Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey

TE3001B
Fundamentación de robótica

Reporte final del reto

Alumnos:

Josue Ureña Valencia	IRS | A01738940
César Arellano Arellano	IRS | A00839373
Jose Eduardo Sanchez Martinez	IRS | A01738476
Rafael André Gamiz Salazar	IRS | A00838280

Docente:
Luis Ricardo Salgado Garza

Fecha de entrega:

13 de Marzo del 2026

Índice

1. Introducción.....2
2. Metodología.....2
2.1. Materiales y componentes requeridos.....2
2.2. Diagrama de bloques de componentes de hardware.....3
2.3. Diagrama de bloques de componentes de software.....3
2.4. Estrategia de control que se implementó.....4
3. Implementación.....4
3.1. Diagrama de conexiones del hardware.....4
3.2. Descripción técnica de los componentes del hardware.....5
3.3. Estrategia de control implementada (PID y PWM).....5
3.4. Descripción del software.....6
3.5. Explicación por bloques del código del microcontrolador.....7
3.6. Protocolo de comunicación con ROS.....11
4. Resultados.....11
4.1. Experimentos realizados.....11
Señal senoidal.....12
Señal cuadrada.....12
Señal escalón.....12
Señal escalón con perturbaciones.....12
4.2. Gráficas de desempeño del sistema.....12
Señal senoidal.....12
Señal cuadrada.....13
Señal escalón.....13
Señal escalón con perturbaciones.....13
5. Conclusiones.....14
6. Referencias.....15

1. Introducción

El control de velocidad de motores DC es un problema fundamental en robótica, automatización y sistemas embebidos. Un motor que opera en lazo abierto no puede garantizar que se mantenga la velocidad deseada, ya que las variaciones de carga, voltaje y fricción afectan su comportamiento. Por esta razón, las estrategias de control de lazo cerrado se utilizan comúnmente para regular la velocidad del motor con precisión.

Para esta entrega final del reto, el objetivo es diseñar e implementar un sistema de control de velocidad de lazo cerrado para un motor DC. El sistema debe regular la velocidad del motor según una señal de referencia generada por ROS, mientras estima la velocidad real del motor mediante la retroalimentación del encoder. Para lograrlo, la solución integra componentes de hardware y software, incluyendo un microcontrolador ESP32, un controlador de motor L298N, un motor DC con encoder y nodos ROS para la entrada de comandos y la monitorización.

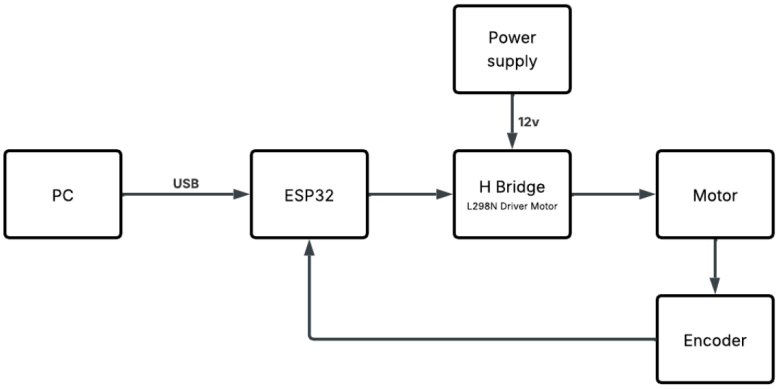
2. Metodología

2.1. Materiales y componentes requeridos

Se incluye una tabla con todos los componentes utilizados en la implementación (hardware y software), indicando nombre, modelo, especificaciones relevantes y cantidad.

Componente	Modelo / Tipo	Especificaciones Relevantes	Cantidad
Microcontrolador	ESP32 Development Board	CPU 240 MHz, WiFi/Bluetooth integrado, salidas PWM, soporte para comunicación serial	1
Motor DC con Encoder	JGA25-370	Encoder cuadratura integrado para medición de velocidad, 140 RPM - 12V	1
Driver de motor	L298N Motor Driver Module	Driver puente H dual, control mediante PWM, hasta 2A por canal	1
Fuente de alimentación	Fuente DC externa	Proporciona el voltaje y corriente necesarios para el motor	1
Protoboard	Protoboard estándar	Utilizada para el montaje y conexión del circuito	1
Cables de conexión	Jumpers (macho-macho / macho-hembra)	Utilizados para realizar las conexiones eléctricas entre componentes	Varios
Computadora	Laptop (Ubuntu)	Ejecuta el entorno ROS y permite la supervisión del sistema	1
Middleware	ROS2 (Robot Operating System)	Permite la comunicación entre nodos y el microcontrolador	1
Entorno de programación	Arduino IDE / micro-ROS	Utilizado para programar el ESP32 y manejar la comunicación con ROS	1

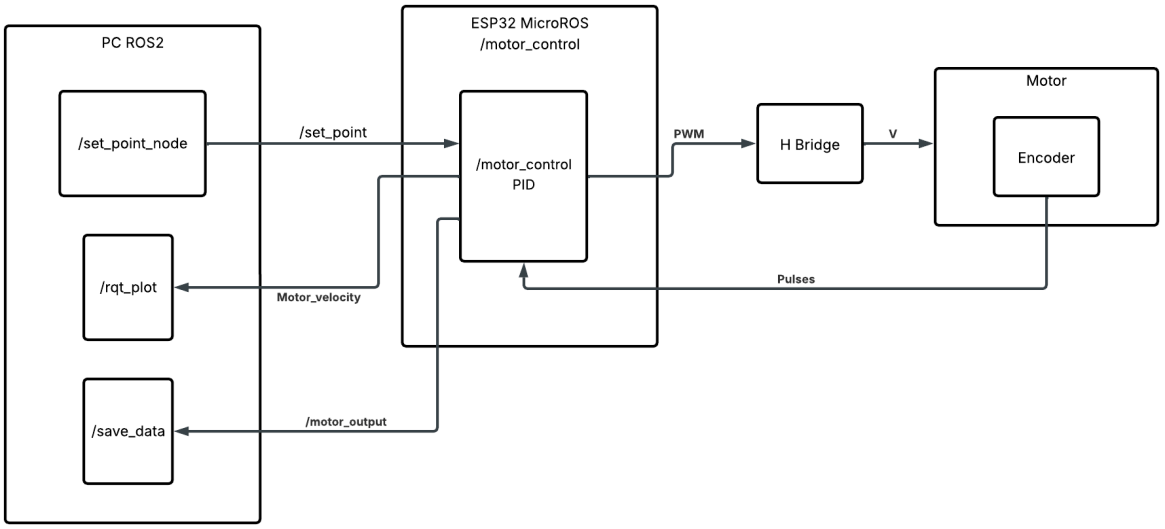
2.2. Diagrama de bloques de componentes de hardware



El diagrama de bloques muestra la arquitectura general del sistema de hardware. La computadora envía las referencias de control al microcontrolador ESP32 mediante comunicación USB. El ESP32 procesa esta información y genera la señal de control que es enviada al driver L298N (puente H), el cual alimenta y controla el motor DC utilizando 12v suministrados por la fuente de alimentación. Finalmente, el encoder mide la velocidad del motor y envía la retroalimentación al microcontrolador, permitiendo implementar el sistema de control en lazo cerrado.

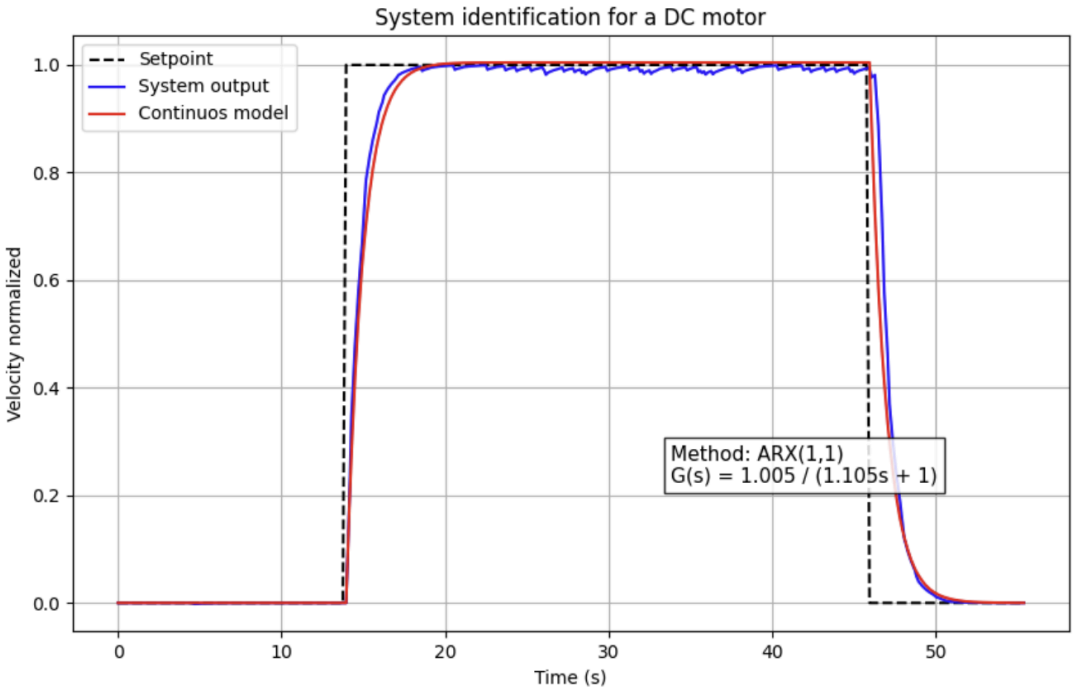
2.3. Diagrama de bloques de componentes de software

El diagrama de bloques de software muestra la interacción entre los módulos que se ejecutan en ROS2 y aquellos implementados en el ESP32 con micro-ROS. En la computadora, el nodo /input_node genera la señal de referencia y la publica en el tópico /set_point. Esta referencia es recibida por el ESP32, donde se ejecuta el controlador PID, el cual calcula la señal de control que se convierte en PWM para accionar el motor mediante el puente H. El encoder proporciona la velocidad medida del motor, la cual es utilizada como retroalimentación para el controlador y también es enviada nuevamente al sistema ROS para su visualización mediante rqt_plot.



2.4. Estrategia de control que se implementó

La imagen muestra la base del ajuste de nuestro controlador. Primero, identificamos la dinámica del motor a partir de su respuesta al escalón. Mediante una aproximación ARX, obtuvimos un modelo de primer orden dado por G(s) = 1.005 / (1.105 s + 1). Con base en esta respuesta, propusimos las ganancias del controlador, mismas que no cumplieron con la respuesta deseada, por lo que a partir de ellas, se realizó un tuneo de manera manual hasta llegar a las siguientes ganancias para la implementación final Kp = 1.6, Kd = 0.02 y Ki = 0.6.

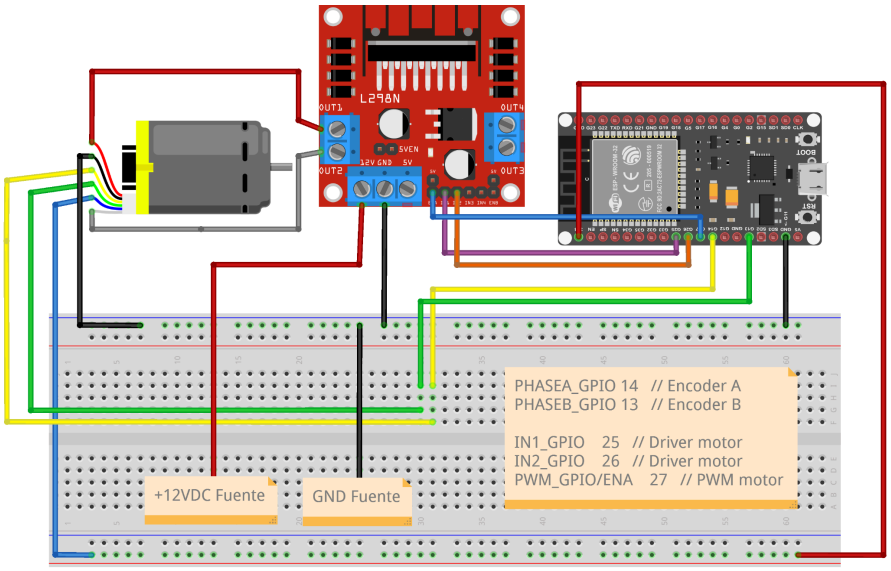


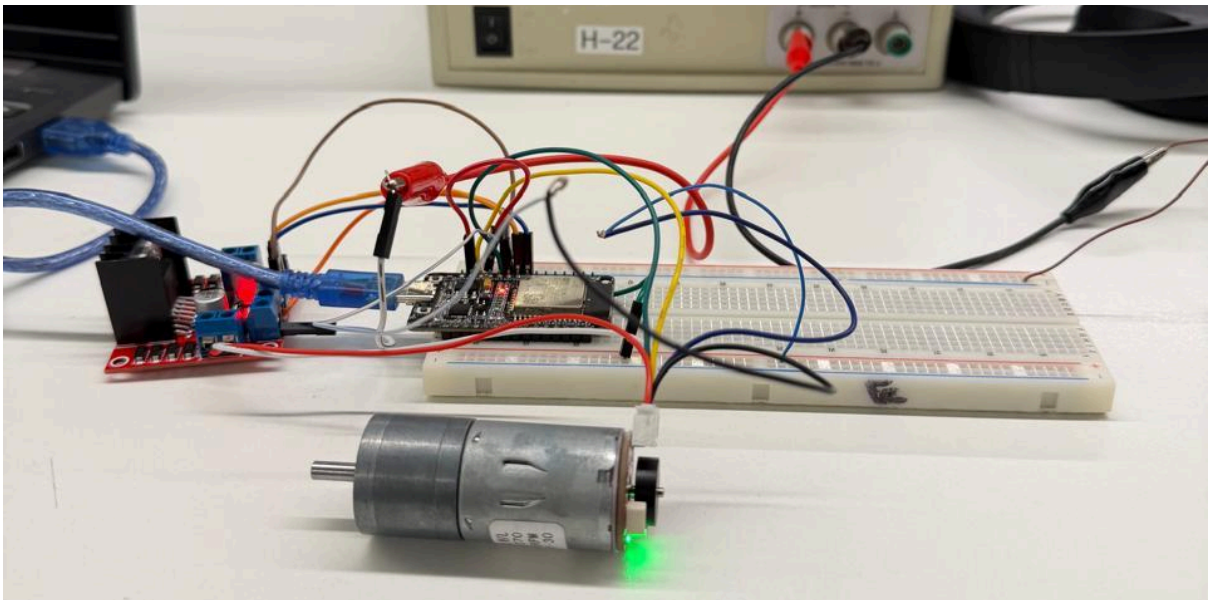
La señal de control resultante se satura y se convierte en PWM para el controlador del motor.

3. Implementación

3.1. Diagrama de conexiones del hardware

Diagrama esquemático de las conexiones de los componentes del sistema junto con la indicación de los pines utilizados para nuestro sistema..





3.2. Descripción técnica de los componentes del hardware

Se describen las características técnicas de los principales componentes utilizados (microcontrolador, motor DC, puente H, driver, encoder, sensores, etc.).

- Microcontrolador (ESP32).

Se utilizó un microcontrolador ESP32, el cual integra un procesador de 32 bits, módulos PWM, temporizadores y comunicación serial. En este proyecto se encarga de ejecutar el controlador PID, procesar la señal del encoder y generar la señal de control para el driver del motor.

- Motor DC.

El actuador principal del sistema es un motor de corriente directa, empleado como planta del proceso. Su velocidad puede regularse variando el voltaje promedio aplicado, lo cual se logra mediante una señal PWM.

- Encoder.

El motor cuenta con un encoder incremental que genera pulsos proporcionales al giro del eje. Esta señal permite estimar la velocidad angular del motor y proporcionar la retroalimentación necesaria para el control en lazo cerrado.

- Driver / Puente H (L298N).

Para accionar el motor se utilizó un módulo L298N, basado en un puente H dual. Este componente permite controlar tanto el sentido de giro como la velocidad del motor a partir de señales digitales y PWM enviadas por el microcontrolador.

3.3. Estrategia de control implementada (PID y PWM)

La estrategia de control implementada se basa en un controlador PID en lazo cerrado, ejecutado en el microcontrolador ESP32. El sistema recibe desde ROS una señal de referencia de velocidad, la cual se compara

con la velocidad real del motor estimada a partir de los pulsos del encoder. Con esta comparación se obtiene el error de control, que es procesado mediante un algoritmo PID incremental.

La salida del controlador corresponde a una señal de control normalizada, la cual se limita dentro de un rango definido para evitar saturaciones. Posteriormente, esta señal se convierte en un valor de PWM, utilizado para regular el voltaje promedio aplicado al motor a través del módulo L298N. De esta manera, el PWM permite ajustar la velocidad del actuador de forma proporcional a la acción calculada por el PID.

Además, el sentido de giro del motor se define mediante las entradas digitales del puente H, mientras que el ciclo de trabajo del PWM determina la magnitud de la acción de control. Finalmente, la velocidad medida se publica nuevamente hacia ROS para monitoreo y retroalimentación del sistema.

3.4. Descripción del software

El software del sistema se divide en dos partes principales: el programa ejecutado en el microcontrolador ESP32 y los nodos del sistema ROS que interactúan con el controlador.

Software del microcontrolador

El microcontrolador ejecuta un programa desarrollado en C/C++ utilizando la librería micro-ROS, que permite integrar el dispositivo como un nodo dentro del ecosistema ROS. En este programa se implementa la lectura del encoder mediante interrupciones, el cálculo de la velocidad del motor y el algoritmo de control PID encargado de regular la velocidad.

El ESP32 recibe la señal de referencia a través de un suscriptor al tópico set_point, donde se envían los valores de velocidad deseada. A partir de la señal del encoder se calcula la velocidad real del motor, la cual se compara con la referencia para obtener el error del sistema. Con este error se calcula la acción de control mediante el controlador PID, cuya salida se convierte en una señal PWM que regula la velocidad del motor a través del driver L298N. Finalmente, el valor de velocidad estimado se publica nuevamente en el tópico motor_output, permitiendo monitorear el comportamiento del sistema desde ROS.

Software del sistema ROS

En el sistema ROS se implementó un nodo encargado de generar la señal de referencia del controlador. Este nodo publica periódicamente valores en el tópico set_point, los cuales pueden corresponder a diferentes tipos de señales de prueba, como senoidal, cuadrada, triangular o escalón, permitiendo evaluar el desempeño del sistema de control bajo diferentes condiciones.

El nodo utiliza temporizadores para generar las señales de referencia de forma continua y publicarlas a una frecuencia definida. De esta manera, el sistema ROS funciona como la interfaz de alto nivel que envía comandos al microcontrolador y recibe la velocidad medida del motor para su análisis y visualización.

3.5. Explicación por bloques del código del microcontrolador

El software del microcontrolador se encuentra organizado en diferentes bloques funcionales que permiten gestionar la comunicación con ROS, adquirir la velocidad del motor mediante el encoder y ejecutar el algoritmo de control.

- Inicialización de librerías y variables:

En esta sección se incluyen las librerías necesarias para el funcionamiento del sistema, incluyendo micro-ROS, así como las definiciones de variables globales y constantes utilizadas por el controlador.

```
C/C++
#include <micro_ros_arduino.h>    //micro-ros-arduino library
#include <rcl/rcl.h>               //Core ROS 2 Client Library (RCL) for node
management.
#include <rcl/error_handling.h>   //Error handling utilities for Micro-ROS.
#include <rclc/rclc.h>               //Micro-ROS Client library for embedded
devices.
#include <rclc/executor.h>        //Micro-ROS Executor to manage callbacks
#include <std_msgs/msg/float32.h>   //Predefined ROS 2 message type
#include  <rmw_microros/rmw_microros.h>  //ROS  Middleware  for  Micro-ROS,
provides functions for interfacing Micro-ROS with DDS.
```

Estas librerías permiten que el microcontrolador funcione como un nodo dentro del sistema ROS, habilitando la comunicación mediante publicadores y suscriptores.

- Definición de pines y parámetros del sistema:

Esta parte corresponde a la configuración de pines físicos y constantes del sistema, donde se establecen las conexiones con el encoder, el puente H y la salida PWM, así como los parámetros nominales del motor y del controlador.

```
C/C++
#define PHASEA_GPIO 14
#define PHASEB_GPIO 13
#define IN1 25
#define IN2 26
#define PWM_PIN 27

#define RPM_MAX 134.0
#define PULSES_PER_REV 495
```

```
#define PWM_FRQ 980
#define PWM_RES 8
#define PWM_MAX 255
#define PWM_CHNL 0
#define MSG_MIN_VAL -1
#define MSG_MAX_VAL 1
#define PWM_CHNL 0
```

También se establecen parámetros importantes como la resolución del encoder y la frecuencia de la señal PWM utilizada para controlar el motor. En esta sección se vincula directamente el software con el hardware físico implementado. Los pines PHASEA_GPIO y PHASEB_GPIO se utilizan para leer las señales del encoder incremental, mientras que IN1, IN2 y PWM_PIN controlan el módulo L298N. Por otro lado, constantes como RPM_MAX y PULSES_PER_REV permiten convertir los pulsos del encoder en una estimación de velocidad en RPM, mientras que los parámetros de PWM definen la frecuencia, resolución y rango de la señal que excita al motor. Esta parte del programa es fundamental porque fija las condiciones con las que el controlador interactúa con la planta real.

- Comunicación con ROS:

El microcontrolador crea un nodo ROS que permite recibir la referencia de velocidad y publicar la velocidad medida del motor.

```
C/C++
 // create node
  RCCHECK(rclc_node_init_default(&node, "motor_control", "", &support));


  // create subscriber
  RCCHECK(rclc_subscription_init_default(
    &subscriber,
    &node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Float32),
    "set_point"));


  RCCHECK(rclc_publisher_init_default(
    &publisher,
```

```
    &node,
    ROSIDL_GET_MSG_TYPE_SUPPORT(std_msgs, msg, Float32),
    "motor_output"));
```

El suscriptor recibe la señal de referencia enviada desde ROS a través del tópico set_point.

- Lectura del encoder:

La medición de la velocidad del motor se realiza mediante una interrupción asociada al encoder.

```C/C++
// ISR Encoder
int dir = 1;
void IRAM_ATTR isr() {
  if(digitalRead(PHASEB_GPIO)){
    dir = 1;
  }else{
    dir = -1;
  }countPhaseA++;
}
```

Esta rutina incrementa el contador de pulsos cada vez que se detecta un cambio en la señal del encoder.

- Cálculo de velocidad y filtrado:

Una vez acumulados los pulsos del encoder durante el tiempo de muestreo, el programa calcula la velocidad en RPM, la limita y aplica un filtrado exponencial para suavizar la señal.

```C/C++
 noInterrupts();
 pulseCount = countPhaseA;
 countPhaseA = 0;
 interrupts();

  // RPM
  rpm_raw = (pulseCount * 60.0) / (PULSES_PER_REV * Ts);
  rpm_raw = clampf(rpm_raw, 0.0, RPM_MAX);
  rpm_filt = alphaRPM * rpm_raw + (1.0 - alphaRPM) * rpm_filt;
  rpm = rpm_filt / RPM_MAX;
```

Primero se copian los pulsos acumulados y se reinicia el contador para evitar inconsistencias por interrupciones concurrentes. Después, la expresión matemática convierte la cantidad de pulsos en revoluciones por minuto usando la resolución del encoder y el tiempo de muestreo. Finalmente, se aplica un filtro de suavizado con factor alphaRPM, lo que reduce el ruido de la medición y hace más estable la respuesta del controlador, especialmente cuando la señal del encoder presenta pequeñas variaciones.

- Implementación del controlador PID:

El controlador PID se implementa en forma incremental para calcular la acción de control a partir del error entre la referencia y la velocidad medida.

```
C/C++
  //PID incremental
  controlSignal = controlSignal
    + Kp * (e0 -e1)
    + Ki * Ts * e0
    + (Kd / Ts) * (e0 -2.0 * e1 + e2);
```

Este algoritmo permite ajustar continuamente la señal de control para minimizar el error del sistema. La ventaja de esta formulación es que corrige la acción previa agregando los aportes proporcional, integral y derivativo. En este caso, el término proporcional responde al cambio inmediato del error, el integral permite corregir error en estado estacionario y el derivativo ayuda a amortiguar variaciones bruscas, mejorando la estabilidad dinámica del sistema.

- Generación de la señal PWM:

La señal de control obtenida del PID se convierte en un ciclo de trabajo PWM que regula la velocidad del motor.

```
C/C++
  //CONVERTIR A PWM
  uint32_t duty = (uint32_t)(controlSignal * PWM_MAX);
  ledcWrite(PWM_CHNL, duty);
```

El ciclo de trabajo del PWM determina el voltaje promedio aplicado al motor a través del driver L298N.

- Ejecución del control:

El algoritmo de control se ejecuta periódicamente dentro del ciclo principal del programa.

```C/C++
  EXECUTE_EVERY_N_MS(100,control_function());
```

De esta forma se garantiza que el cálculo de velocidad, el PID y la actualización del PWM se ejecuten de manera periódica durante el funcionamiento del sistema.

3.6. Protocolo de comunicación con ROS

La comunicación entre el microcontrolador y el sistema ROS se realiza mediante micro-ROS, el cual permite integrar dispositivos embebidos como nodos dentro de la arquitectura de ROS 2. En este sistema, el microcontrolador ESP32 se comunica con un agente micro-ROS que se ejecuta en la computadora donde está instalado ROS, utilizando una conexión serial.

Dentro de esta arquitectura, el ESP32 actúa como un nodo ROS denominado motor_control, el cual se encarga de recibir la referencia de velocidad y enviar la velocidad medida del motor. Para ello se implementa un suscriptor al tópico set_point, que recibe los valores de referencia generados desde ROS, y un publicador al tópico motor_output, que transmite la velocidad estimada del motor a la computadora para su monitoreo.

Por otro lado, en el sistema ROS se implementa un nodo adicional encargado de generar la señal de referencia del controlador. Este nodo publica periódicamente valores en el tópico set_point, utilizando diferentes tipos de señales de prueba como senoidal, cuadrada, triangular o escalón, lo que permite evaluar el desempeño del sistema de control bajo distintas condiciones de operación.

De esta forma, el flujo de comunicación del sistema se establece de la siguiente manera: el nodo de ROS genera la señal de referencia y la publica en el tópico set_point; el microcontrolador recibe esta referencia, ejecuta el controlador PID y regula la velocidad del motor mediante PWM; posteriormente, la velocidad medida se publica en el tópico motor_output, permitiendo visualizar y analizar el comportamiento del sistema desde ROS.

4. Resultados

4.1. Experimentos realizados

Con el objetivo de evaluar el desempeño del sistema de control implementado, se realizaron cuatro experimentos utilizando diferentes señales de referencia generadas desde el sistema ROS. Estos experimentos permitieron analizar la capacidad del controlador para seguir la referencia y su comportamiento ante cambios y perturbaciones externas.

Señal senoidal

En el primer experimento se aplicó una señal de referencia senoidal. Este caso de prueba permitió evaluar la capacidad del controlador para seguir referencias que varían de manera continua en el tiempo y observar el comportamiento del sistema ante cambios suaves en la velocidad deseada.

Señal cuadrada

En el segundo experimento se utilizó una señal de referencia cuadrada. Esta señal presenta cambios abruptos entre dos niveles de velocidad, lo que permitió analizar la respuesta dinámica del sistema y la capacidad del controlador para adaptarse rápidamente a cambios repentinos en la referencia.

Señal escalón

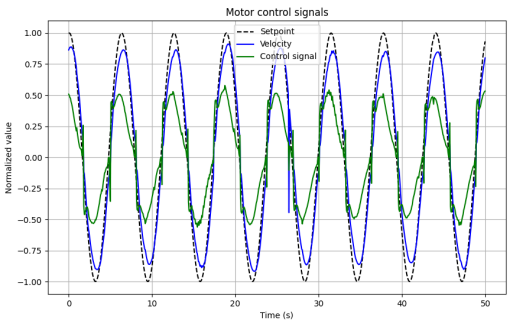
El tercer experimento consistió en aplicar una señal de tipo escalón. Este caso de prueba se utilizó para evaluar la respuesta transitoria del sistema, observando características como el tiempo de establecimiento y la capacidad del controlador para alcanzar y mantener la velocidad deseada.

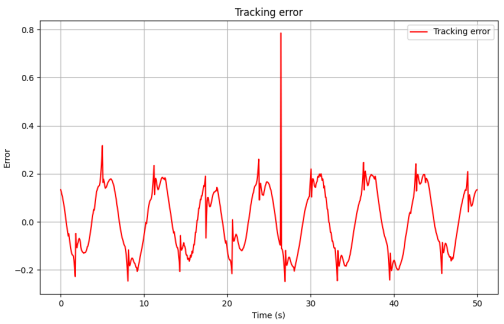
Señal escalón con perturbaciones

Finalmente, se realizó una prueba utilizando una señal escalón mientras se introducían perturbaciones externas al sistema. Estas perturbaciones se generaron aplicando resistencia manual al eje de salida del motor. Este experimento permitió evaluar la robustez del controlador y su capacidad para compensar perturbaciones y recuperar la velocidad de referencia.

4.2. Gráficas de desempeño del sistema

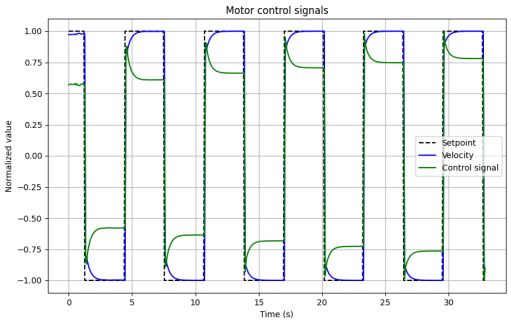
Señal senoidal

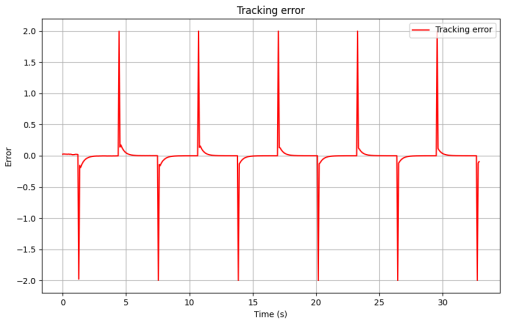




En esta prueba se demostró que el controlador sigue exitosamente la referencia con la velocidad medida muy de cerca al setpoint durante todo el experimento, manteniendo el error dentro de ±0.2, con una anomalía detectada cerca de t=25s asociada a una perturbación momentánea durante un cambio de dirección, siendo la condición más exigente para el controlador. El error no crece con el tiempo, confirmando ser un sistema estable.

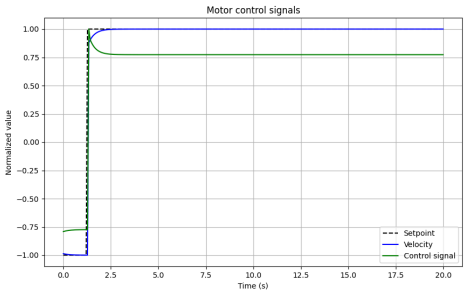
Señal cuadrada

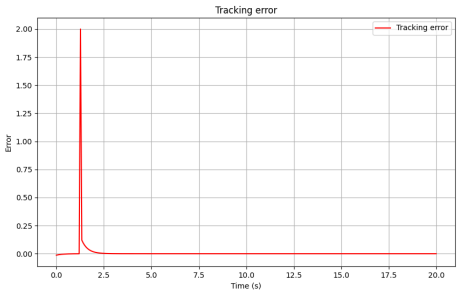




La prueba con señal cuadrada es la más exigente del sistema, ya que exige cambios muy bruscos de velocidad entre +1 y -1. La velocidad medida sigue el setpoint con un tiempo de establecimiento rápido en marco estacionario, alcanzando el valor de referencia en cada ciclo. Los picos en el error de seguimiento de hasta ±2.0 ocurren únicamente en las transiciones, donde el motor debe invertir dirección completamente, sin embargo, esto es esperado y refleja la dinámica mecánica del motor más que una limitación del controlador. Una vez superada la transición, el error regresa a cero rápidamente, confirmando que el sistema es estable y el controlador es capaz de rechazar el error en estado estacionario.

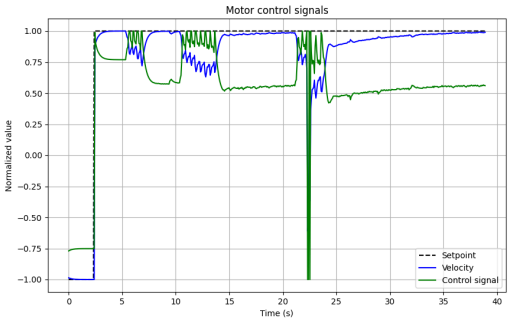
Señal escalón

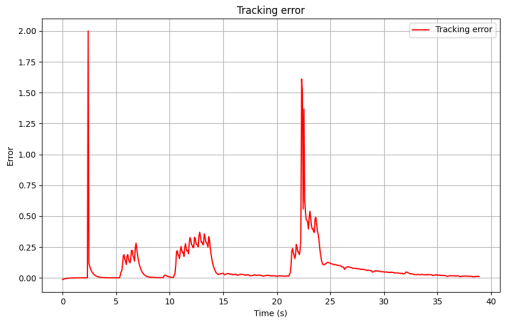




En esta prueba se demostró que el controlador responde exitosamente ante una señal escalón, siguiendo la referencia de forma rápida y precisa desde el momento en que el setpoint cambia de -1.0 a +1.0, con la velocidad medida estabilizándose en el valor deseado en cuestión de segundos.

Señal escalón con perturbaciones





Por último, se demostró que el controlador responde exitosamente ante una señal escalón con perturbaciones externas, perturbaciones que podemos observar durante el intervalo entre t = 5 s y t = 20 s, las perturbaciones

aplicadas manualmente al eje del motor generan oscilaciones mostrando al controlador intentando compensar la resistencia externa.

5. Conclusiones

El sistema de control de velocidad implementado demostró ser funcional y robusto en todos los escenarios de prueba. La integración entre el ESP32, el driver L298N y el entorno ROS2 mediante micro-ROS resultó efectiva, permitiendo un flujo de comunicación claro y una arquitectura bien definida entre hardware y software.

El controlador PID incremental, logró seguir correctamente las cuatro señales de referencia evaluadas. En condiciones de estado estacionario, el error se mantuvo cercano a cero en todos los casos. Las mayores exigencias ocurrieron con la señal cuadrada, donde los cambios abruptos entre +1 y -1 generaron picos de error de hasta ±2.0 durante las transiciones; sin embargo, estos picos se deben a la inercia mecánica del motor y no representan una deficiencia del controlador, ya que el sistema se recupera rápidamente. La prueba con perturbaciones externas confirmó la capacidad de rechazo del controlador: ante la resistencia aplicada manualmente al eje, el sistema compensa la perturbación y retomó el setpoint, aunque con oscilaciones visibles que evidencian que el término derivativo podría beneficiarse de un filtro para reducir el ruido amplificado.

Un aspecto a mejorar es la estimación de velocidad. El cálculo basado en conteo de pulsos por periodo fijo funciona bien a velocidades medias, pero puede volverse ruidoso o poco preciso en velocidades bajas o durante cambios bruscos de dirección, como se observó en la anomalía cerca de t = 25 s en la prueba senoidal. Una alternativa más robusta sería el método de medición por tiempo entre pulsos, que ofrece mejor resolución a bajas RPM.

También vale la pena considerar la saturación de la señal de control. Aunque se implementó para evitar que el PWM salga del rango válido, no se integró mecanismo anti-windup en el término integral, lo que puede provocar sobreimpulso o respuesta lenta tras periodos prolongados de saturación. Incorporarlo sería una mejora directa sobre el desempeño observado.

En términos de arquitectura, el diseño modular del software, tanto en el ESP32 como en los nodos de ROS, facilita la escalabilidad del sistema. Sería relativamente sencillo extender el proyecto a control de posición o a sistemas con múltiples motores sin rediseñarlo desde cero.

En conclusión, el reto cumplió con sus objetivos, se implementó un sistema de control en lazo cerrado funcional, respaldado por identificación de sistema, con resultados experimentales que validan su desempeño. Las áreas de mejora identificadas son puntuales y atacables, lo que refleja un diseño sólido con espacio claro para refinamiento.

6. Referencias

[1] C. F. Pardo Martín, "Método de Ziegler-Nichols," Picuino, ene. 2026. [En línea]. Disponible en: https://www.picuino.com/es/control-ziegler-nichols.html

[2] Manchester Robotics Ltd., *"TE3001B Intelligent Robotics Implementation 2026."* GitHub repository. [En línea]. Disponible en: https://github.com/ManchesterRoboticsLtd/TE3001B_Intelligent_Robotics_Implementation_2026